EXCEL-to-EDB Converter

This document describes the Microsoft Excel -based format that is accepted by EEvision's `excel2edb` converter program. This Excel format is a simplified data input format to EEvision, but can only define a subset of EDB's capabilities.

The Excel file should consist of a single table that describes all relevant data. We assume that the first row of this table contains the table headers, which are evaluated by the conversion tool. In general, every (other) row of the table corresponds to one wire of the harness.

The description of a wire encompasses five parts:

1. The actual *wire information* like wire ID, wire name, wire type (like power, ground, bus etc.), and an arbitrary number of attributes of the wire.
2. *Multicore information* (whether the wire is twisted or shielded together with other wires), and attributes of the multicore. We also support nested multicores where, e.g., bundles of wires are twisted and then shielded together.
3. The *first connection point* of the wire, i.e., cavity, connector, and component to which one end of the wire is connected. The headers of all columns that belong to this group all start with the prefix "**A-**".
4. The same information for the *second connection point*. The headers of all columns that belong to this group start with the prefix "**B-**".
5. *Module information* specifies to which signal, harness, bus, or function a wire, component, connector, or cavity belongs.

No-wire entries: There is an exception to the rule that each line creates one wire. For instance, if the system contains components with cavities that are not connected to a wire or nested multicores that do not directly contain wires, but only other multicores. In this case, the wire information is left empty and only the relevant part is filled with data. The described objects are created but without connection to a wire. These no-wire entries have an empty Wire ID and also the other wire fields like WireType and wire attributes must be empty.

Case sensitive: The reserved column headers are case insensitive (so writing "Wire", "wire" or "WIRE" is equivalent). In contrast, attribute names and the contents of the data cells are case sensitive.

A missing column has the same meaning as if all its fields were empty. The Name columns are an exception to this rule as described below.

Comment Columns

All columns with either an empty header or a header that starts with the character "#" are considered comment columns and ignored by the conversion tool.

Object Identifiers

Every object that should be created needs an object identifier. The IDs are used to refer to objects, e.g., at an inliner component to specify which connectors are attached to each other. We require that all object IDs are unique within their container. Here are examples for IDs:

Wire	Multicore	A-Comp	A-Conn	A-Cav	B-Comp	B-Conn	B-Cav
W4711	TW1	8276	cc	1	1241	A	7

Wire: `W4711`
Multicore: `TW1`
Components: `8276` and `1241`
Connectors: `8276.cc` and `1241.A`
Cavities: `8276.cc.1` and `1241.A.7`

Object Names

Apart from an ID, each object can have a name that can be displayed in EEvision. The name can either be identical to the ID, or a separate name (which does not need to be unique) can be specified. The name of an object is obtained as follows:

- If the table contains a column “Name”, then the entry in this column specifies the displayed name of the wire (it can also be the empty string when the table cell is empty; in this case, no name will be displayed). If there is no column “Name”, then the Wire ID is used as the wire name.
- Similar: A-CompName, B-CompName (for components), A-ConnName, B-ConnName (for connectors), A-CavName, B-CavName (for cavities), MCName (for multicores), SignalName (for the signal module), HarnessName (for the harness module), and BusName (for the bus module).

Object Attributes

Every object in the EDB can carry an arbitrary number of attributes. All columns whose names start with “Wire:” are turned into attributes of the wire (don't confuse “Wire” with “Wire:”). The field “Wire:xyz” defines the wire attribute “xyz”. The same applies to other objects' attributes, like “A-Comp:xyz” or “A-Conn:xyz” or “A-Cav:xyz” etc.

There is one reserved attribute name that is interpreted by the converter tool: The attribute “ color” (for wires specified in the column “Wire: color”, for components in “A-comp: color” – note the leading space in the attribute name!) contains color information. It is expected to be a space-separated list of color values. These can either be given in hexadecimal RGB notation (like “#FF0000” for red) or using the color keywords from the SVG standard like “red”, “green” etc. Color names and hexadecimal RGB values can be mixed. The value of “ color” attributes is case-insensitive.

Arbitrary Column Header: Additionally, all columns whose headers do not correspond to one of the reserved identifiers are also treated as wire attributes. This means “Wire:width” and “width” have identical meanings. If a wire attribute needs to be created whose name starts with “#”, then the form with prefix “Wire:” needs to be used; otherwise the column is treated as a comment column.

Overview on the Column Headers

Object	ID	Name	Type		Attribute N	Attribute M
Wire	**Wire**	**Name**	**Type**		**Wire:N**	**M** (if no conflict)

Component	**A-Comp** **B-Comp**	**A-CompName** **B-CompName**	**A-CompType** **B-CompType**		**A-Comp:N** **B-Comp:N**	**A-Comp:M** **B-Comp:M**
Connector	**A-Conn** **B-Conn**	**A-ConnName** **B-ConnName**	**A-ConnType** **B-ConnType**		**A-Conn:N** **B-Conn:N**	**A-Conn:M** **B-Conn:M**
Cavity	**A-Cav** **B-Cav**	**A-CavName** **B-CavName**	**A-CavType** **B-CavType**		**A-Cav:N** **B-Cav:N**	**A-Cav:M** **B-Cav:M**
Multicore	**MC**	**MCName**	**MCType**	**MCParent**	**MC:N**	**MC:M**
Module	**Harness**	**HarnessName**			**Harness:N**	**Harness:M**
	Signal	**SignalName**			**Signal:N**	**Signal:M**
	Bus	**BusName**			**Bus:N**	**Bus:M**

Repeated Values: Object values do not need to be repeated. For instance, if several wires connect to the same component, the component appears in the table for each of these wires. In order to avoid redundancy, it suffices to specify the name, type and attribute value of the component once at its first occurrence; the remaining entries can be left empty. However, if a value is repeated, all occurrences must be consistent; otherwise a warning is printed. Which value is taken is implementation defined. Actually all fields except the Object IDs can be left empty at the repeated occurrences. The first occurrence (that must have all values defined) is the first from top, then within a row the leftmost one.

Empty Values: If an entry in an attribute column is empty, then this attribute is not defined for that object. This also means, there is no way to define empty-value attributes.

Example:

Wire	**Name**	**Type**	**Wire: color**	**Wire:Diameter**
w12	+12V	POWER	#FF0000	0.5mm

This example describes a wire with ID `w12` without any connections, but with name “+12V” (can be displayed at the wire), with type, with reserved attribute “ color” (can be displayed in a small rectangle at the wire) and with general purpose attribute “Diameter”. Note that instead of “#FF0000” it is also possible to write “red” for the value of the color attribute.

Wire Types: In the EDB data model, one can distinguish between different types of wires like power and ground wires. This information is specified in the “Type” column. Possible values for Type (case insensitive, empty string is allowed) are:

- POWER
- GROUND
- LOGICAL
- BUS
- HV
- ARC
- UNDEF (default, equivalent to the empty string)

Wire connections: By default, each wire connects to two cavities. However, also fewer and more connections are supported. If the wire connects only to zero (“unconnected wire”) or one cavity (“wire stub”), either the entries for the A-side or the B-side can remain empty or even both. If the wire connects to n > 2 cavities, the wire ID is repeated in (at least) [n/2] rows. Up to two connections per row can be specified.

Wires of type ARC play a special role. They do not represent physical wires, but instead model electrical connections within a component between its cavities. For instance, the electrical connection between the

two pins of a fuse can be modeled using an ARC wire. Arc wires are not directly displayed in EEvision, but taken into account by the different analysis algorithms, e.g., when showing the electrically connected wires in “Net Mode”.

Example:

Wire	A-Comp	A-Conn	A-Cav	B-Comp	B-Conn	B-Cav
w732	M23	A	2	D42	B	2
w732	M1	A	2			

This example describes a wire with ID `w732` that is connected to the three cavities `M23.A.2`, `D42.B.2` and `M1.A.2`.

Components

Component Types: We need to distinguish between the different possible types of components. The type is specified in the column A-CompType. Possible values are (case insensitive):

- ECU
- INLINER
- SVG
- UNDEF (default, same as empty string)

ECU

Component – Example:

A-Comp	A-CompName	A-Comp:Part No	A-CompType	A-Comp: color
A40	Brake Ctrl	N239	ECU	#ffffaa

This example creates an ECU component with ID `A40`, name “Brake Ctrl”, body color #ffffaa (yellow), and an attribute “Part No” with value “N239”. Apart from A-Comp, all columns are optional.

Connector – Example:

A-Conn	A-ConnName	A-Conn:Location	A-ConnType
c1	A	+438	MALE

This example creates a connector at the ECU of the current line. The connector has the ID `c1`, the name “A”, which is displayed in EEvision, and an attribute “Location” with value “+438”. The connector is a male connector.

Connector Types: The following values (case insensitive, empty string allowed) are supported:

- MALE
- FEMALE
- INVISIBLE
- UNDEF (default, same as empty string)

If A-Conn is empty (because an ECU without connectors is created), the remaining connector entries must be empty as well.

Cavity – Example:

A-Cav	A-CavName	A-CavType	A-Cav:Description
c1	1	IN	Power Input
c2		HALFDOT,OUT	GND

This example creates a cavity with ID `c1` and name “1”; it is an input to the ECU and has an attribute “Description” with value “Power Input”. Additionally, a cavity with ID `c2` and no name is created; it is an output with a halfdot symbol (the optional IN and OUT will display little wire direction indicators, close to the connection points).

Cavity Types: The following values (case insensitive, empty string allowed) are supported:

- HALFDOT
- SPLICED
- IN
- OUT
- UNDEF (default, same as empty string)

If A-Conn is empty, all cavity entries must be empty. If A-Cav is empty, the remaining cavity entries must be empty as well.

INLINER

Component

An inliner component is created when the A-CompType field is set to INLINER.

Connector

For the connectors, we need to specify the partner connector and a number of flags. This happens in the column A-ConnType. The syntax for this specification is:

```
<comma-separated list of flags>:<partner connector Id>
```

Possible flags are: MALE, FEMALE, ANTI, HALF, UNDEF. ANTI is used for inliners with more than one connector per side. All connectors on one side need to be flagged with ANTI, the connectors on the opposite side without ANTI. The colon is mandatory when a partner connector is specified, even if the list of flags is empty. If there is no partner connector, the colon can be omitted. The flag HALF indicates that there is no partner connector.

Cavity

The same syntax is used for the specification of cavities. Possible flags are SPLICED, HALFDOT, IN, OUT, UNDEF. Note that at most one of SPLICED and HALFDOT and at most one of IN and OUT may be specified. UNDEF is equivalent to specifying no flag.

By default, the partner cavity is the cavity of the partner connector with the same ID. Alternatively, the ID of the partner cavity can be given after the colon. That means a cavity does not have a partner if the partner cavity ID after the colon is missing (i.e., the type field ends with a colon) or the colon is missing and there is no cavity at the partner connector with the same ID. The partner relations on the cavities override the derived partner relation from the connector.

Example:

A-Comp	A-CompType	A-Conn	A-ConnType	A-Cav	A-CavType
inl99	INLINER	J	FEMALE:P	1	IN
inl99	INLINER	J	FEMALE:P	2	OUT

inl99	INLINER	P	MALE:J	1	OUT
inl99	INLINER	P	MALE:J	2	IN

This example creates an Inliner with one connector on each side.
The redundant data is displayed in **gray** color; those fields may be empty instead.

Redundant Data: Each field that defines the information first (from top to bottom, then left to right), is mandatory. At any follow-up field that defines the same information again, must store the identical value or an empty field.

Example:

A-Comp	A-CompType	A-Conn	A-ConnType	A-Cav	A-CavType
inl80	INLINER	J1	FEMALE:P1	1	
inl80	INLINER	P1	MALE,ANTI:J1	1	
inl80	INLINER	J2	FEMALE:P2	1	OUT
inl80	INLINER	P2	MALE,ANTI:J2	1	IN

This example creates an Inliner with two connectors on each side.

Example:

A-Cav	A-CavType	A-Conn	A-ConnType	A-Comp	A-CompName	A-CompType
1	:b	A	female,anti:B	C0		inliner
2	halfdot	A	female,anti:B	C0		inliner
3	in:a	A	female,anti:B	C0		inliner
4	halfdot,out	A	female,anti:B	C0		inliner
5	out	A	female,anti:B	C0		inliner
a	out:3	B	male:A	C0		inliner
b	in:1	B	male:A	C0		inliner
c	halfdot,out	B	male:A	C0		inliner
d	halfdot	B	male:A	C0		inliner
e	halfdot	B	male:A	C0		inliner
5	in	B	male:A	C0		inliner

This example creates an Inliner with paired connectors A and B and partly paired cavities.

This example pairs the cavities `A.1` ⇆ `B.b`, `A.3` ⇆ `B.a`, and `A.5` ⇆ `B.5`. The first two pairs are defined by the :b and :a notation in the A-CavType column, but the last pair is defined just by the identical cavity IDs `5` in the A-Cav column.

Splices and Eyelets

Component

Splices and Eyelets are modeled as connectors only, i. e., all fields in the component columns must be empty (this is a difference to the EDB data model).

Connector

The **Connector Type** must be set to SPLICE or to EYELET in order to create a Splice or Eyelet object. Each splice and eyelet is identified by a unique connector ID in the A-Conn or B-Conn column. These

IDs form one global name-space for all splices and one for all eyelets.

Example:

Wire	A-Comp	A-Conn	A-ConnType	A-Cav
w11		S121	SPLICE	
w12		E248	EYELET	
w13		E248	EYELET	1
w14		E248	EYELET	1

This example creates a splice (with ID `S121`) and an eyelet (with ID `E248`) and four wires connecting to them.

Cavity

Splices must have no cavity information, i. e., all fields in the cavity columns should be empty. Eyelets may have no cavity information or may optionally specify cavity information to bundle connecting wires (by multi-term connections on the same cavity).

Multicores

Example:

Wire	MC	MCType	MCParent	MC:Cover
w11	TS1	TWSHIELDED	S	green
w12	TS1	TWSHIELDED	S	green
w21	TS2	TWSHIELDED	S	blue
w22	TS2	TWSHIELDED	S	blue
w31	TS3	TWSHIELDED	S	red
w32	TS3	TWSHIELDED	S	red
w41	TS4	TWSHIELDED	S	black
w42	TS4	TWSHIELDED	S	black
	S	SHIELDED		orange
wsh	S	SHIELD		

This example defines twisting and shielding for an Ethernet cable; and adds a general attribute “Cover”, stored in the column “MC:Cover” (here, it denotes the color of the shield cover). It defines the multicores with the IDs `TS1`, `TS2`, `TS3`, `TS4` nested inside the multicore with the ID `S`.

The twisting and shielding is defined as a tree, with multicores as the nodes and wires as the leaves. The “MCParent” column defines the nesting hierarchy by referring to other multicores by ID.

Multicore Types: The following values for “MCType” (case insensitive, empty string allowed) are supported:

- TWISTED
- SHIELDED
- TWSHIELDED
- UNDEF (default, equivalent to the empty string)

defining a twisting node, shielding node or both, twisting and shielding in one node. An empty value (or UNDEF) defines a logical grouping without any graphical indicators. The special value “SHIELD” just adds a shield connector wire to a SHIELDED or TWSHIELDED node.

Modules

Modules are used to group related database objects. For example, all wires carrying the same signal can be included in a module. EEvision supports different types of modules: Harness modules for grouping wires and their cavities as well as connectors into harnesses; signal modules for wires that carry a certain signal; bus modules for wires of the same bus; function modules to group wires by functions; and other modules that cannot be defined by this Excel data format.

Apart from columns describing module attributes, there can be up to three columns per module type: For instance, for harness modules, the column “Harness” contains the ID of the harness module, the column “HarnessName” specifies the name of the module if it is different from the ID. Finally, “HarnessMask” defines which objects of the current row (wire, multicore, component, connector, and cavity for both the first and second connection) should be contained in the module.

Replacing “Harness” by “Signal”, “Bus”, or “Func” yields signal, bus, or function modules.

We support two different notations for the contents of the “HarnessMask” column. The first one is a comma-separated list of tags, the other one a hexadecimal value, obtained as the sum of the tags' values. The available tags are:

Tag	Wire	MC	A-Comp	A-Conn	A-Cav	B-Comp	B-Conn	B-Cav
Value	0x0001	0x0002	0x0010	0x0020	0x0040	0x0100	0x0200	0x0400

If the mask column is missing or the table cell empty, a default value is used, which is supposed to cover the most common cases. For the different module types we use the following default values:

- *Signal modules* are expected to contain only wires. Therefore the default for “SignalMask” is “Wire” (= 0x0001).
- The same holds for *bus modules*. Therefore we also use “Wire” (0x0001) as the default value.
- *Harness modules* typically contain all elements of a cable harness, which are typically wires the cavities and the connectors they are connected to. This includes splices and eyelets, but not the components. Therefore the default value is “Wire,A-Conn,A-Cav,B-Conn,B-Cav” (= 0x0661).
- *Function modules* typically not only contain the harness elements, but also the components. Accordingly, we set the default to “Wire,A-Comp,A-Conn,A-Cav,B-Comp,B-Conn,B-Cav” (= 0x0771).

Signal – Example:

Wire	**Signal**	**SignalName**	**Signal:Description**
W3491	Kl.31	K31	Ground
W3498	Kl.31	K31	Ground

This example defines two wires that belong to the same signal with the ID `Kl.31`, name “K31”, and a general attribute “Description”.

Each wire typically belongs to only one signal module (or to no module, represented by an empty field in the “Signal” column).

Bus – Example:

Wire	**Bus**	**BusName**	**Bus:Spec**	**Bus:Term**

W701	CAN4	CAN Multimedia	ISO 11898-2	150 Ohm
W702	CAN4	CAN Multimedia	ISO 11898-2	150 Ohm

This example defines two wires that belong to the same bus with the ID `CAN4` and name “CAN Multimedia” and two general attributes “Spec” and “Term”.

Harness – Example:

Wire	A-Comp	A-Conn	A-Cav	B-Comp	B-Conn	B-Cav	Harness	HarnessMask	Harness:Supplier
W237	A56	J1	1	S23	P3	7	H1277		LEONI AG
W250	A56	J1	2	T1	J2	5	H1277	Wire,A-Conn,A-Cav	LEONI AG

This example defines two wires that belong to the same harness with the ID `H1277` and adds a general attribute “Supplier” to that harness module. Apart from the two wires, the harness module also contains the connectors `J1` of component `A56` together with its cavities `1` and `2`, and the connector `P3` of component `S23` with the cavity `7`. Note that `T1`'s connector `J2` and its cavity `5` are not members of the harness module because of the explicitly specified HarnessMask. Specifying 0x0061 instead of “Wire,A-Conn,A-Cav” would have the same effect.

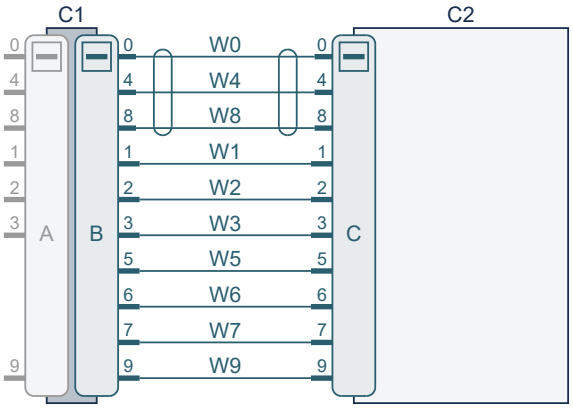
Function – Example:

Wire	A-Comp	A-Conn	A-ConnType	A-Cav	B-Comp	B-Conn	B-Cav	Func	FuncName	FuncMask
W123		SP12	Splice		A24	P	1	F012	Front Light	
W123								F017	Back light	0x0001

This table defines two function modules: “Front light” and “Back light”. The wire is a member of both of them. Additionally, “Front Light” contains the splice SP12 and the component A24 with its connector P and the cavity 1.

Example tapp1

Here is a Excel-based table (as Tapp1.csv and Tapp1.xlsx) that stores the same example design as created by the Tcl example program tapp1.tcl (or the Java example program Tapp1.java, or the Python example program Tapp1.py). The schematic diagram, as created by EEvision, looks as follows:

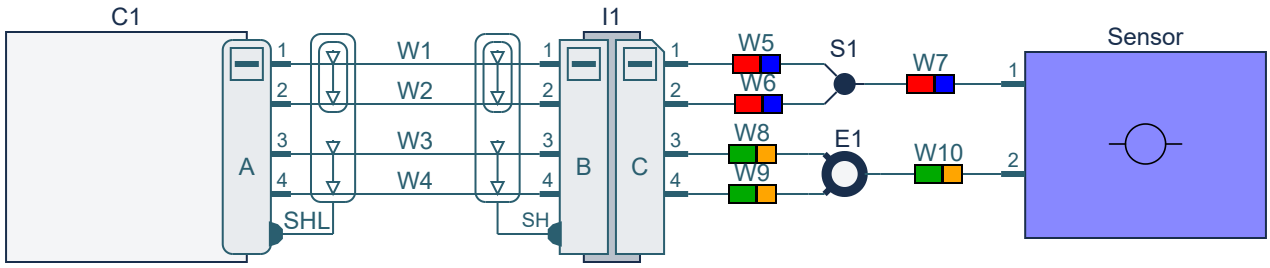


Wire	A-Cav	A-CavType	A-Conn	A-ConnType	A-Comp	A-CompType	B-Cav	B-Conn	B-Comp	B-CompType	MC	MCTyp

W0	0		B	:A	C1	inliner	0	C	C2	ecu	mc	shield
W1	1		B		C1		1	C	C2			
W2	2		B		C1		2	C	C2			
W3	3		B		C1		3	C	C2			
W4	4		B		C1		4	C	C2		mc	
W5	5	:	B		C1		5	C	C2			
W6	6	:	B		C1		6	C	C2			
W7	7	:	B		C1		7	C	C2			
W8	8		B		C1		8	C	C2		mc	
W9	9		B		C1		9	C	C2			
	0		A	:B	C1							
	1		A		C1							
	2		A		C1							
	3		A		C1							
	4		A		C1							
	8		A		C1							
	9		A		C1							

Example tapp3

Here is a Excel-based table (as Tapp3.csv and Tapp3.xlsx) that stores the same example design as created by the Tcl example program tapp3.tcl (or the Java example program Tapp3.java, or the Python example program Tapp3.py). The schematic diagram, as created by EEvision, looks as follows:



Wire	Wire: color	A-Cav	A-CavName	A-CavType	A-Conn	A-ConnName	A-ConnType	A-Comp	A-CompType	B-Cav	B-
W1		1	1		A	A		C1	ecu	1	1
W2		2	2		A			C1		2	2
W3		3	3		A			C1		3	3
W4		4	4		A			C1		4	4
SHL		Y4		halfdot	A			C1		Y4	
W5	red blue	1	1		C	C	male:B	I1			
W6	red blue	2	2		C			I1			
W7	red blue				S1					1	1

W8	#00aa00 #ffa500	3	3		C			I1		Y0	
W9	#00aa00 #ffa500	4	4		C			I1		Y1	
W10	#00aa00 #ffa500	Y2			E1					2	2